Software Test Design

White box vs. Black box testing

Black-box testing

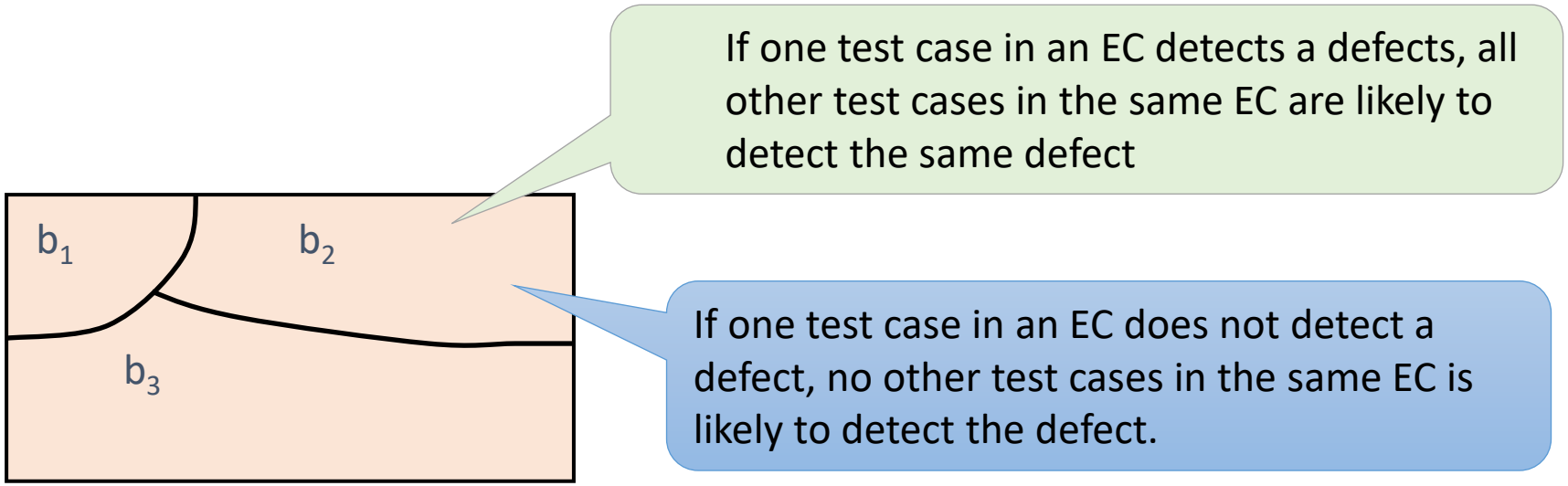
- Aka specification testing
- Test case generation are based solely on the knowledge of the system requirements.
- Ensures that specified features of the software are addressed by some aspect of the program

White-box testing

- Aka Code-based testing
- Test data selection is guided by information derived from the internal structure and internal functional properties of the program

Equivalence Class Testing

- Goal: to reduce the number of test cases
- Sub-divide the set of all possible inputs into equivalence classes (EC) that receive equivalent treatment
- Select one test case from each partition



Example

Input variable
Deposit Amount

Valid ECs:
EC1: $4,999.99 or below
EC2: $5,000.00 to $499,999.99
EC3: $500,000.00 to $999,999.99
EC4: $1,000,000.00 or above

Invalid ECs:
IEC1: $0 or below

Hong Kong Dollar Deposit Rates

HKD Savings Rate

Deposit Balance (HKD)	Interest Rate (p.a.)
$1,000,000.00 or above	0.0010%
$500,000.00 to $999,999.99	0.0010%
$5,000.00 to $499,999.99	0.0010%
$4,999.99 or below	0.0000%

VIP i-Account / YOU i-Account / Other Integrated Account[6] Multi-Currency Current Account Deposit Rate

Deposit Balance (HKD)	Interest Rate (p.a.)
$1,000,000.00 or above	0.0010%
$500,000.00 to $999,999.99	0.0010%
$5,000.00 to $499,999.99	0.0010%
$4,999.99 or below	0.0000%

https://www.dahsing.com/pws/rate-enquiry/hkd?lang=en-US

Test Case	Valid EC	Deposit Amount	Expected output (Interest rate)
TC1	$0.01 to $4,999.99	$1000	0%
TC2	$5,000.00 to $499,999.99	$10,000	0.8750%
TC3	$500,000.00 to $999,999.99	$600,000	0.8750%
TC4	$1,000,000.00 or above	$2,000,000	0.8750%

Test Case	Invalid EC	Deposit Amount	Expected output
TC1	0 or below	0	ERROR

Boundary Value Testing (BVT)

- Errors tend to occur near the extreme values of an input variable
- Examples of test cases
  - Just below minimum (min-)
  - Minimum (min)
  - Nominal value
  - Maximum (max)
  - Just above maximum (max+)

EC2: $5,000.00 to $499,999.99

Test Cases

- $4999.99
- $5,000.00
- $10,000
- $499,999.99
- $500,000.00

Types of Fault related to BVT

- Closure fault
  - The relational operator ≤ is implemented as <
  - The relational operator < is implemented as ≤.
- Shifted-Boundary fault
  - When the constant term of the inequality defining the boundary takes up a value different from the intended value
  - E.g. x>5 becomes x>4

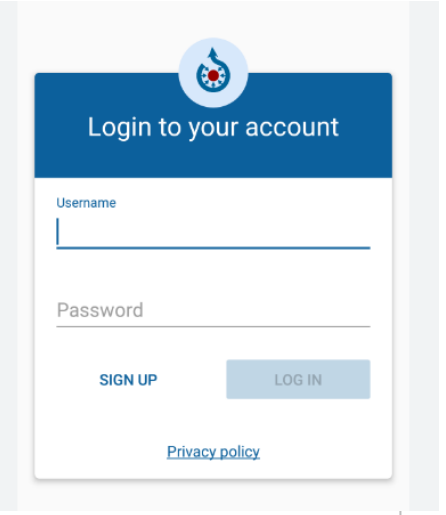
Test Case	EC	Deposit Amount	Expected output (Interest rate)
TC1	$0.01 to $4,999.99	$1000	0%
TC2	$5,000.00 to $499,999.99	$10,000	0.8750%
TC3	$500,000.00 to $999,999.99	$600,000	0.8750%
TC4	$1,000,000.00 or above	$2,000,000	0.8750%

```
def calculate_interest(balance):
    if balance >= 1000000:
        interest_rate = 0.00875
    elif balance >= 500000:
        interest_rate = 0.00875
    elif balance >= 5000:
        interest_rate = 0.00875
    else:
        interest_rate = 0.0

    interest = balance * interest_rate
    return interest

interest_earned = calculate_interest(balance)
print("Interest earned: $", interest_earned)
```

Test Coverage

- Helps answer the following questions
  - Is your software tested sufficiently?
  - In what software component you haven't tested sufficiently?
- Black box coverage criterion
  - Requirements coverage: # of tested requirement/total requirements
- White box coverage criterion
  - Statement coverage
  - Decision coverage
  - Condition coverage
  - Multiple condition coverage
  - Modified condition/Decision coverage (MC/DC)

Example: Requirement Coverage



Each requirement should be covered by at least ONE test case.

1. Requirement: Minimum and Maximum Length
    1. The username field should accept a minimum of 6 characters and a maximum of 20 characters.
    2. The password field should accept a minimum of 8 characters and a maximum of 30 characters.
2. Requirement: Character Types Accepted
    1. The username field should accept alphanumeric characters (A-Z, a-z, 0-9) and special characters such as ".", "_", and "-".
    2. The password field should accept a combination of alphanumeric characters (A-Z, a-z, 0-9) and special characters such as ".", "_", "@", and "#".
3. Requirement: Character Types Not Accepted
    1. The username field should not accept spaces or any other special characters apart from ".", "_", and "-".
    2. The password field should not accept spaces or any special characters other than ".", "_", "@", and "#".
4. Requirement: Password Strength
    1. The password field should enforce a minimum level of complexity by requiring at least one uppercase letter, one lowercase letter, one numeric digit, and one special character.

State transition testing

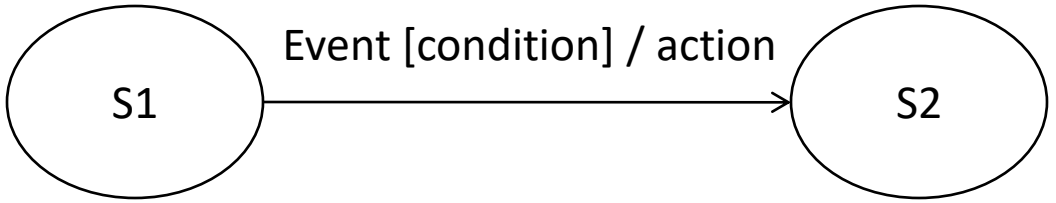
- In many systems, behaviour/output depends on the input as well as the current state
- The current state depends on
  - The initial state
  - The sequence of inputs the system has received in the past
- Example: ON/OFF button, ATM

https://poe.com/wc-TV-state

State transition diagram

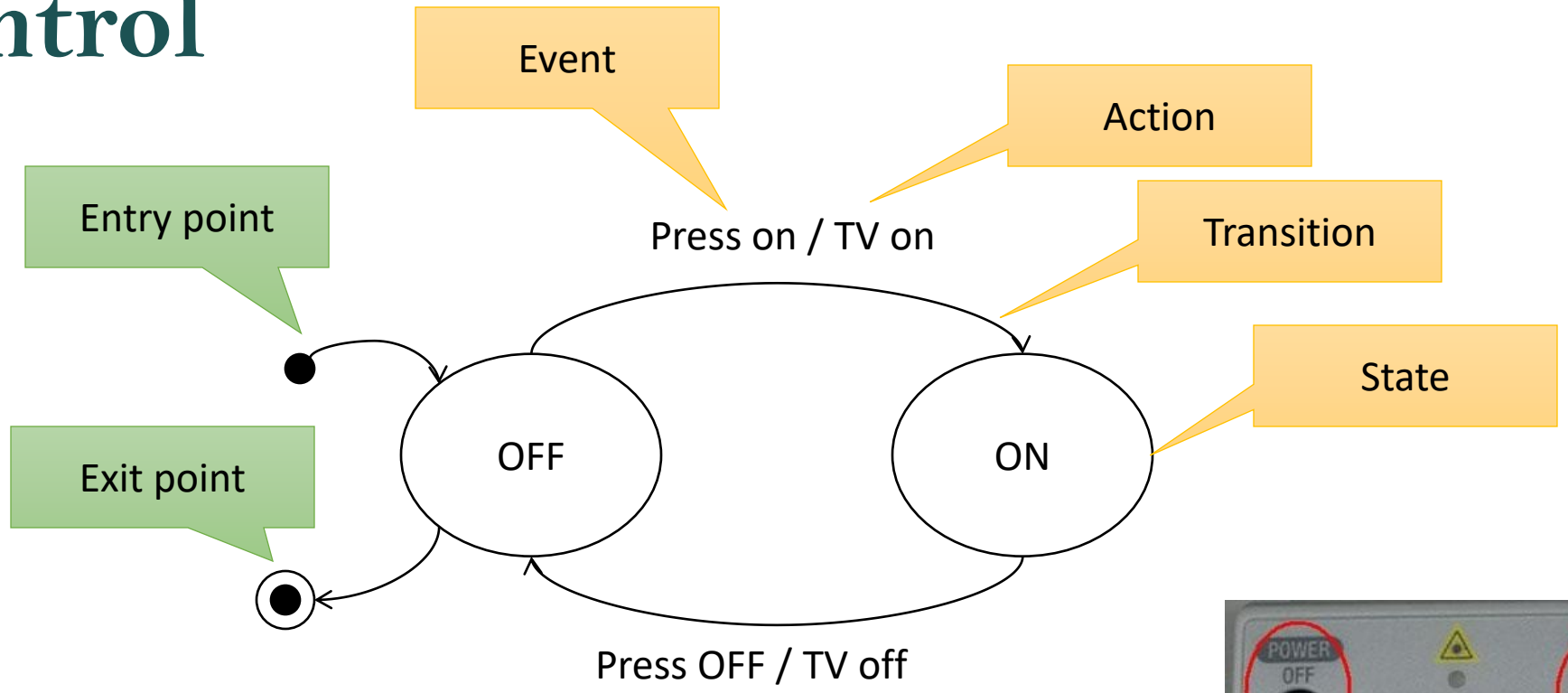
- In state transition testing, test cases are generated from the state-transition diagrams (the specification).
- The diagram documents the states that a system can exist in and the events that come into and are processed by a system as well as the system's responses
- Components
  - State
    - Condition in which a system is waiting for one or more events.
    - Represented by a circle
  - Event
    - May be external (e.g. input by the user) or event generated within the system (e.g. timeout event).
    - With an event, the system can change state or remain in the same state and/or execute an action
  - Action
    - An operation initiated because of a state change
    - E.g. output a certain message, start a timer, generate a ticket

Notation



- Transition (represented by an arrow)
  - represents a change from one state to another
  - each transition can be associated with an event (which triggers the transition) and action (which specify the behaviour/output)
- Entry point (represented as a black dot)
- Exit point (represented as a bulls-eye symbol)

Example: ON/OFF Button for remote control



Test case:
1) (Press ON, Press OFF)

Coverage Criteria for state transition testing

$$\text{state coverage} = \frac{no.\ \text{of states exercised}}{\text{total } no.\ \text{of states}}$$

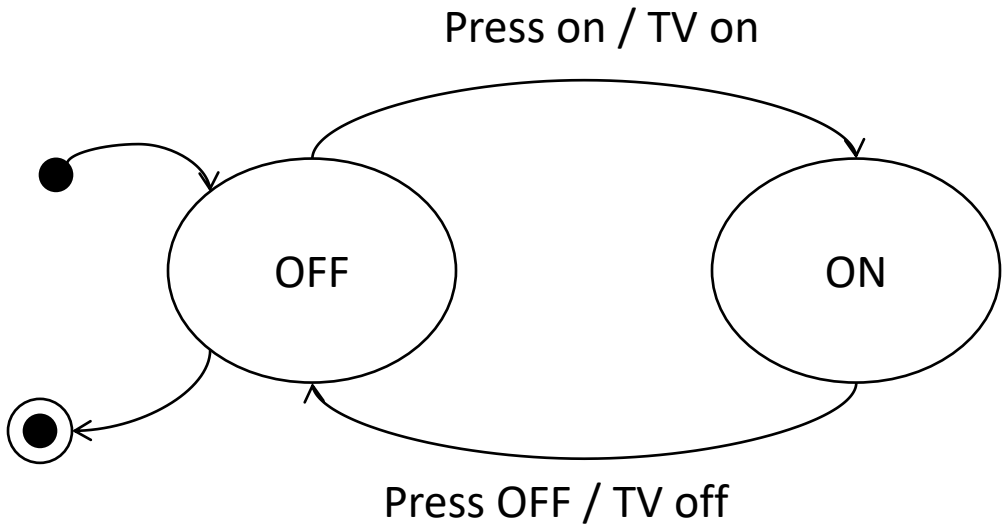
$$\text{transition coverage} = \frac{no.\ \text{of transitions exercised}}{\text{total } no.\ \text{of transitions}}$$

$$2\text{-transition coverage} = \frac{no.\ \text{of sequences of 2 transitions exercised}}{\text{total } no.\ \text{of sequences of 2-transitions}}$$

Note: We can generalize the 2-transition coverage to n transitions coverage.

$$\text{state-input coverage} = \frac{no.\ \text{of state-input pairs exercised}}{no.\ \text{of states} \times no.\ \text{of inputs}}$$

Improved test set

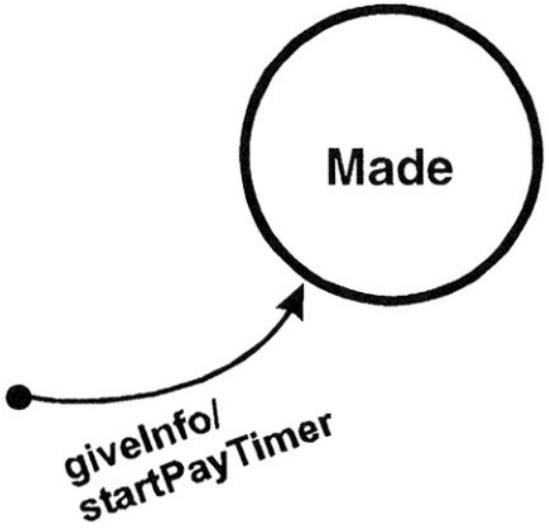


Test case:

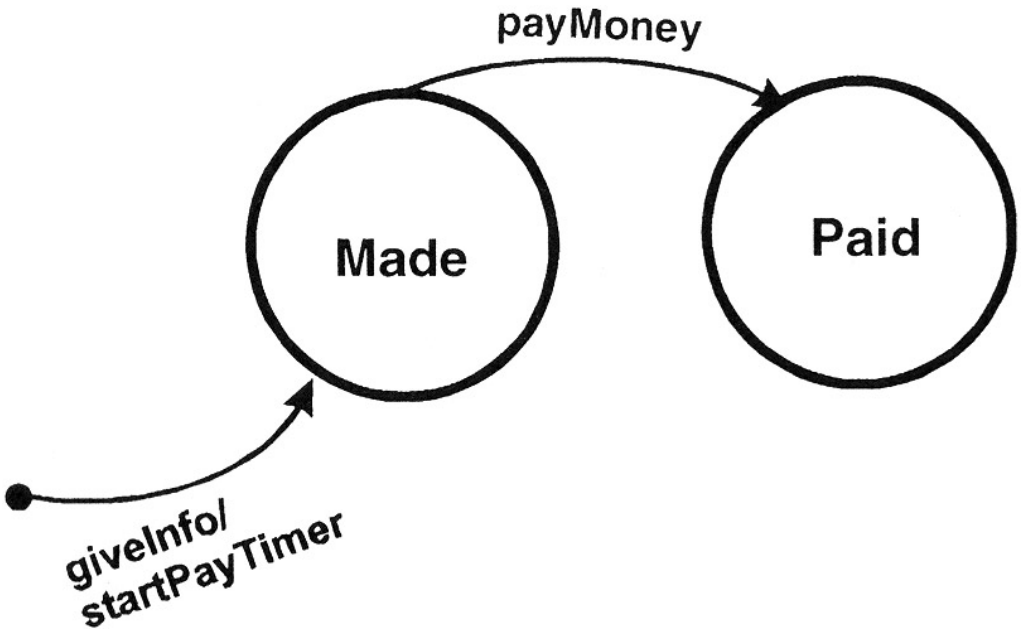
1) (Press ON, Press OFF, Press ON, Press OFF)
2) (Press OFF, Press OFF, Press ON, Press ON, Press OFF)

Example: Airline reservation application

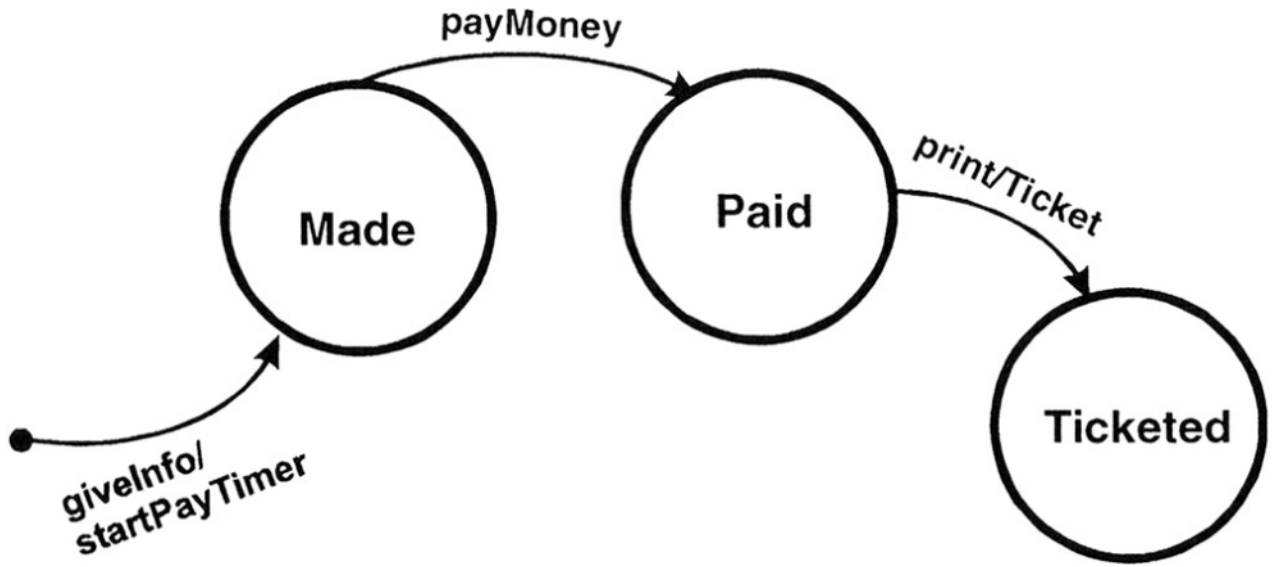
- The customer first provide some information including departure and destination cities, dates, and times, which is used by the reservation agent to make a reservation.
- The reservation is now at the "made" state.



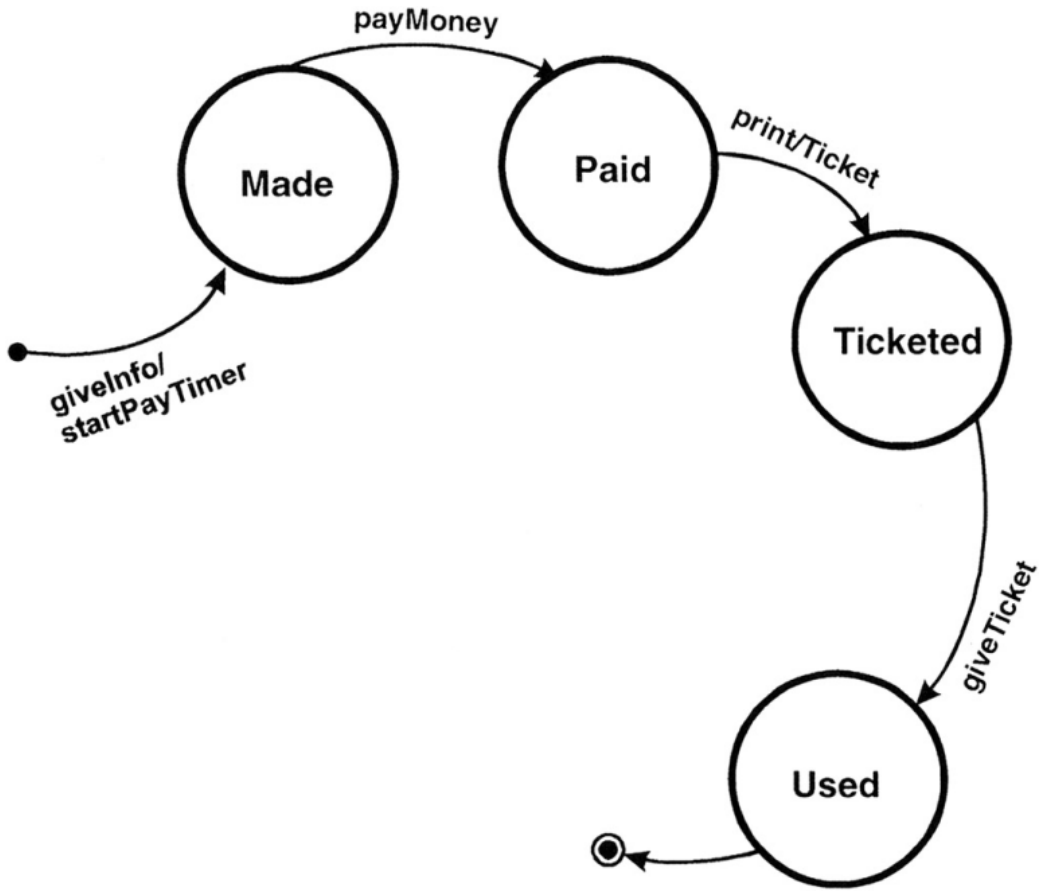
- If the customer pays before the startPayTimer expires, the reservation transits to the "Paid" state.



- From the Paid state the Reservation transitions to the Ticketed state when the print command (an event) is issued.
- In addition to entering the Ticketed state, a Ticket is output by the system.



- From the Ticketed state we giveTicket to the gate agent to board the plane.



Some alternative paths

- If the reservation is not paid for in the time allotted (the PayTimer expires), it is cancelled for non-payment.
- From the Made state the customer (through the reservation agent) asks to cancel the reservation. A new state, Cancelled By Customer, is required.
- In addition, a reservation can be cancelled from the Paid state. In this case a Refund should be generated and leave the system. The resulting state again is Cancelled By Customer.
- From the Ticketed state the customer can cancel the Reservation. The airline will generate a refund but only when it receives the printed ticket from the customer.

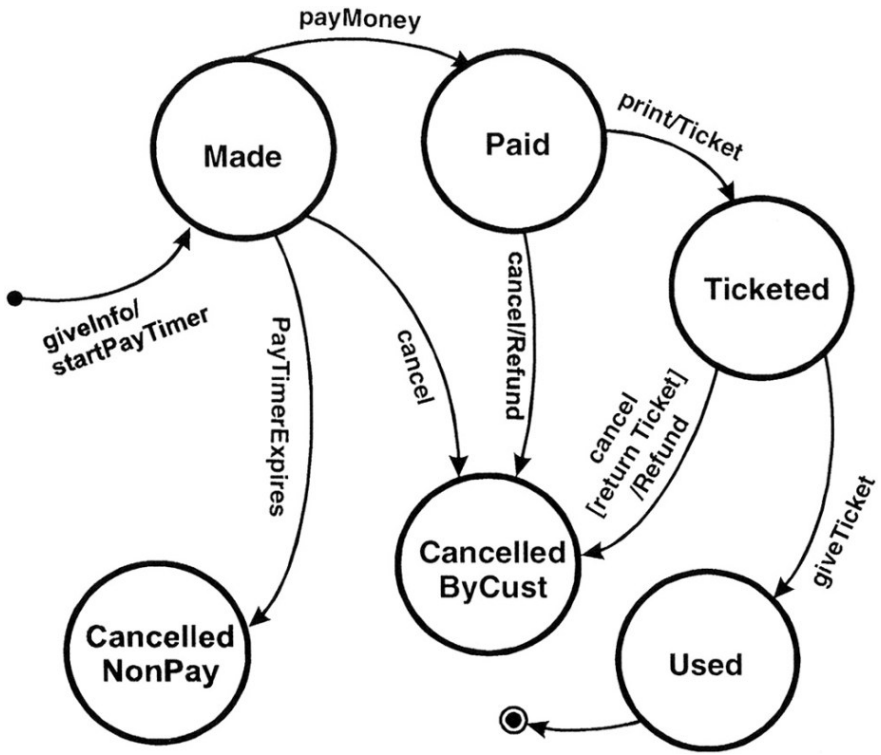
Exercise

Perform state transition testing for the airline reservation application.

What is the minimum number of test cases required to

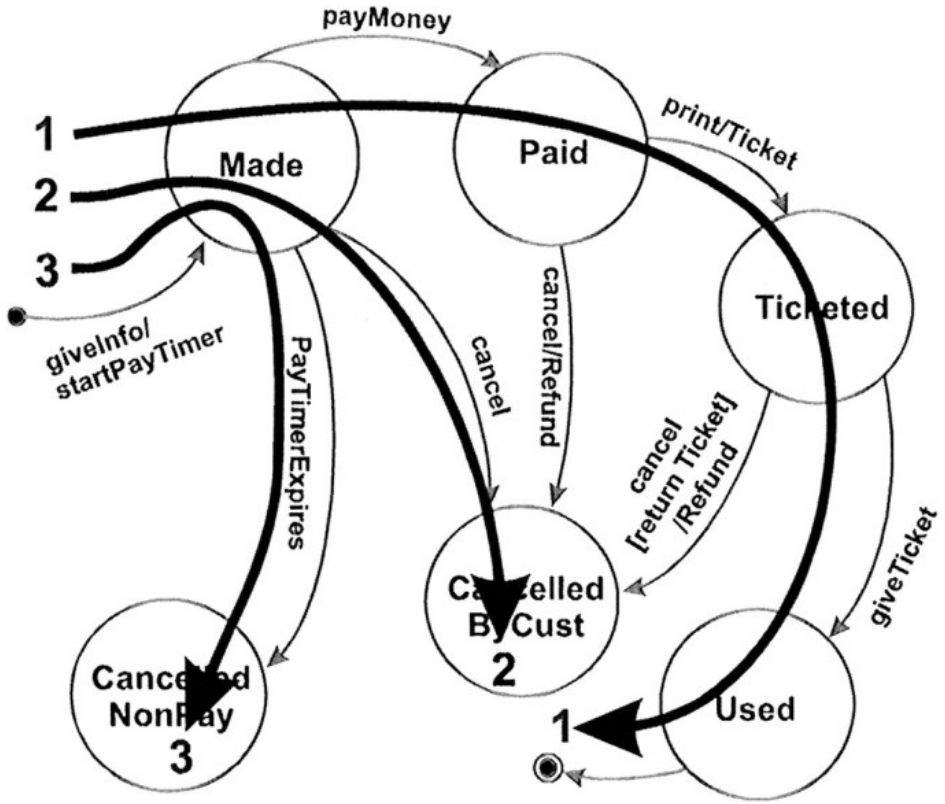
i) Covers all states?

ii) Covers all transitions?



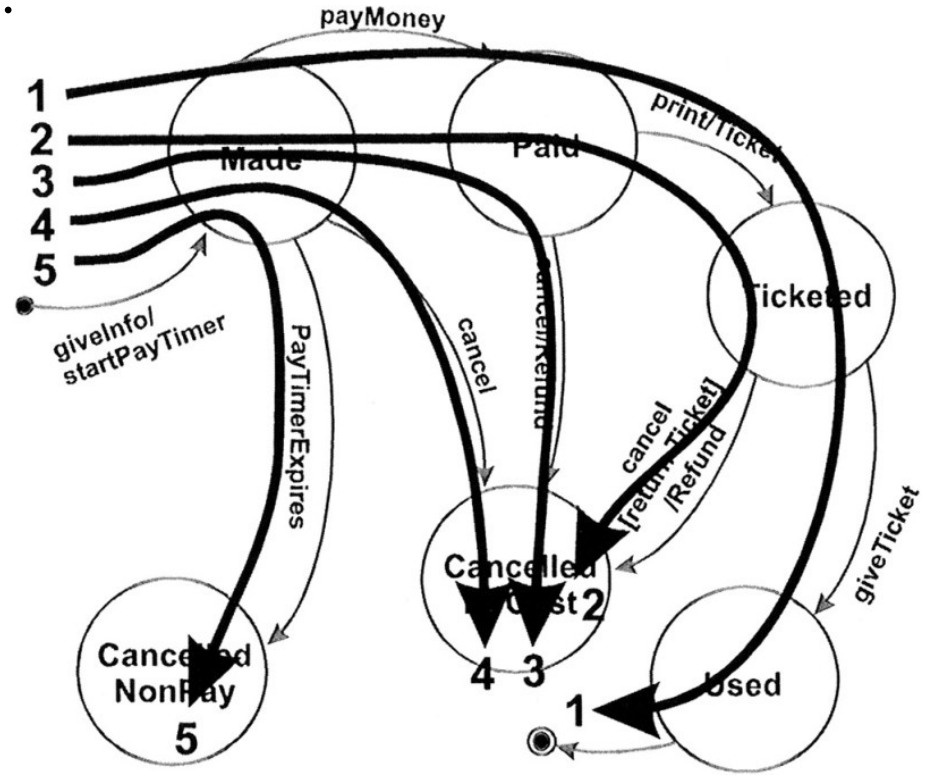
Testing all the states

- Create a set of test cases such that all states are "visited" at least once under test.
- Generally, this is a weak level of test coverage.



Testing all the transitions

- Create a set of test cases such that all transitions are exercised at least once under test.
- This level of testing provides a better level of coverage without generating large numbers of tests.



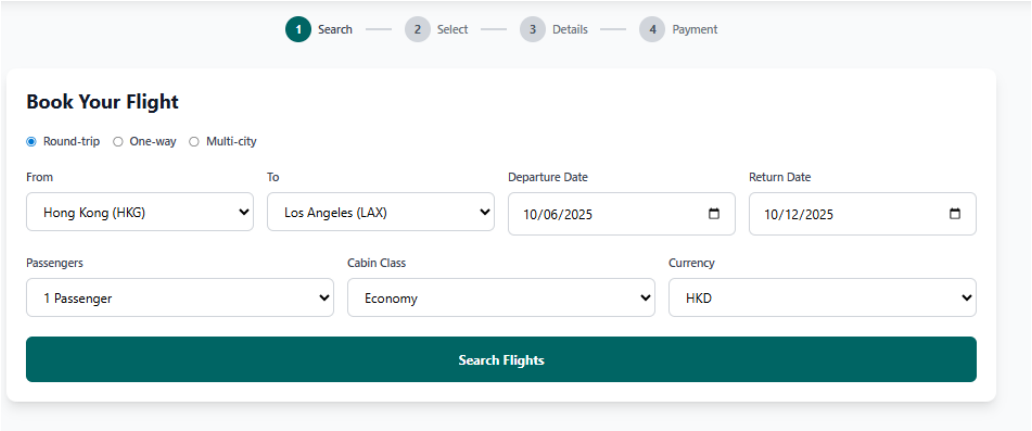
Problem of testing AI generated code

- How do we ensure we sufficiently test every piece of code that is generated by AI, and ensure the AI is not adding extra unwanted features to our codebase?

```
const validateSearch = () => {
  const newErrors = {};
  const now = new Date();
  const maxDeparture = new Date(now);
  maxDeparture.setFullYear(maxDeparture.getFullYear() + 1);
  const dep = new Date(`${searchParams.depDate}T00:00:00`);
  const ret = new Date(`${searchParams.retDate}T00:00:00`);

  if (searchParams.departure === searchParams.destination) newErrors.route = "Identical Cities";
  if (dep <= now) newErrors.depDate = "Departure date must be after current system time.";
  if (dep > maxDeparture) newErrors.depDate = "Departure date must be within 1 year";
  if (searchParams.tripType === 'roundtrip' && ret <= dep) newErrors.retDate = "Return date must be after
  departure date.";
  if (searchParams.tripType === 'roundtrip') {
    const maxReturn = new Date(dep);
    maxReturn.setMonth(maxReturn.getMonth() + 6);
    if (ret > maxReturn) newErrors.retDate = "Return must be within 6 months";
  }

  setErrors(newErrors);
  if (Object.keys(newErrors).length > 0) {
    addLog("Validation Error", JSON.stringify(newErrors));
    return false;
  }
  return true;
};
```

Combination testing

- Evaluates the interactions between different input parameters or components to identify defects caused by their combinations,
- **Factor**: Input variables
- **Level**: Assignable value to a factor
  - The tester may select all or some interesting values using test case selection methods (such as equivalent partitioning, boundary value analysis)



From: {Hong Kong, Tokyo, Singapore}
To: {Los Angeles, New York, London}
Cabin Class: {Economy, Premium Economy, Business, First-Class}

Test Strategy: Each choice

- Each value of each parameter to be included in at least one test case
- Test cases are identified by combining the next unused value of each parameter

```
From:    {Hong Kong, Tokyo, Singapore}
To:      {Los Angeles, New York, London}
Cabin Class: {Economy, Premium Economy, Business, First-Class}
```

Test Case	From	To	Cabin Class	Remarks
TC1	Hong Kong	Los Angeles	Economy	First value from each parameter
TC2	Tokyo	New York	Premium Economy	Next unused value from each param
TC3	Singapore	London	Business	Next unused value from each param
TC4	Hong Kong	Los Angeles	First-Class	Cabin Class last unused value; Reuse From and To

Base Choice

- One best test case is selected
  - Simplest, smallest, or first test case, most likely value (from an end-user point of view)
- New test cases are created by varying the interesting values of one parameter at a time, keeping the values of the other parameters fixed on the base test case.

```
From:    {Hong Kong, Tokyo, Singapore}
To:      {Los Angeles, New York, London}
Cabin Class: {Economy, Premium Economy, Business, First-Class}
```

Test Case	From	To	Cabin Class	Explanation
TC_base	Hong Kong	Los Angeles	Economy	Base case
TC1	Tokyo	Los Angeles	Economy	Vary "From"
TC2	Singapore	Los Angeles	Economy	Vary "From"
TC3	Hong Kong	New York	Economy	Vary "To"
TC4	Hong Kong	London	Economy	Vary "To"
TC5	Hong Kong	Los Angeles	Premium Economy	Vary "Cabin Class"
TC6	Hong Kong	Los Angeles	Business	Vary "Cabin Class"
TC7	Hong Kong	Los Angeles	First-Class	Vary "Cabin Class"

Interaction faults

- Programs often involve the interaction of multiple variables
  - E.g. The display of a character in a word processor depends on the font size, font color, whether it is bolded, whether it is underlined, etc.
- Simple fault:
  - A fault triggered by a single input variable setting.
  - Example: If flight booking crashes whenever the origin is set to "Tokyo," regardless of other inputs
- T-wise (t>=2) interaction fault
  - Faults triggered only when the input variables are set to a specific value

Examples

- 2-wise interaction fault
  - The booking system crashes only when:
    - To = "London“, Cabin Class = "First-Class“
- 3-wise interaction fault
  - The booking system crashes only when the following three input variables are set to these specific values simultaneously:
    - From = "Tokyo“, To = "London“, Cabin Class = "Business“
  - The crash does NOT occur if only one or two of these conditions are met.
  - The fault is triggered only when all three inputs match this combination at the same time.

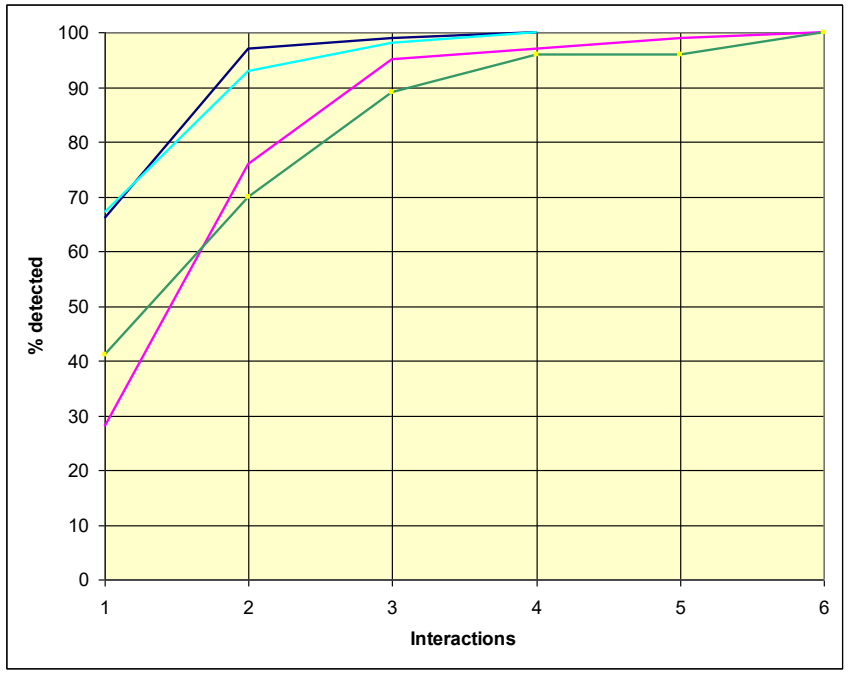
Combinatorial explosion

- The more variables you combine, the higher the number of possible tests
  - The total number of factor combinations is n^k, where k is the number of factors and each factor assumes a value from a set of n values
- Suppose that each factor combination yields one test case.
- If a program has 15 factors with 4 levels each, the total number of tests is $4^{15} \sim 10^9$!

Pairwise Testing

- In pairwise testing, the test cases are generated to cover all combinations of selected test data values for each pairs of variable.
- It is estimated that pairwise testing finds about 50% to 90% of flaws

It is believed that most software defects are not due to complex interaction, but rather interaction of 1 or 2 fields/variables.



Test cases for pairwise testing

From: {Hong Kong, Tokyo, Singapore}
To: {Los Angeles, New York, London}
Cabin Class: {Economy, Premium Economy, Business, First-Class}

Test Case	From	To	Cabin Class	Covered pairs
TC1	Hong Kong	Los Angeles	Economy	Covers (Hong Kong, Los Angeles), (Hong Kong, Economy), (Los Angeles, Economy)
TC2	Tokyo	New York	Premium Economy	Covers (Tokyo, New York), (Tokyo, Premium Economy), (New York, Premium Economy)
TC3	Singapore	London	Business	Covers (Singapore, London), (Singapore, Business), (London, Business)
TC4	Hong Kong	New York	First-Class	Covers (Hong Kong, New York), (Hong Kong, First-Class), (New York, First-Class)
TC5	Tokyo	London	Economy	Covers (Tokyo, London), (Tokyo, Economy), (London, Economy)
TC6	Singapore	Los Angeles	Premium Economy	Covers (Singapore, Los Angeles), (Singapore, Premium Economy), (Los Angeles, Premium Economy)

Parameter 1	Value 1	Parameter 2	Value 2
From	Hong Kong	To	Los Angeles
From	Hong Kong	To	New York
From	Hong Kong	To	London
From	Tokyo	To	Los Angeles
From	Tokyo	To	New York
From	Tokyo	To	London
From	Singapore	To	Los Angeles
From	Singapore	To	New York
From	Singapore	To	London
From	Hong Kong	Cabin Class	Economy
From	Hong Kong	Cabin Class	Premium Economy
From	Hong Kong	Cabin Class	Business
From	Hong Kong	Cabin Class	First-Class
From	Tokyo	Cabin Class	Economy
From	Tokyo	Cabin Class	Premium Economy
From	Tokyo	Cabin Class	Business
From	Tokyo	Cabin Class	First-Class
From	Singapore	Cabin Class	Economy
From	Singapore	Cabin Class	Premium Economy
From	Singapore	Cabin Class	Business
From	Singapore	Cabin Class	First-Class
To	Los Angeles	Cabin Class	Economy
To	Los Angeles	Cabin Class	Premium Economy
To	Los Angeles	Cabin Class	Business
To	Los Angeles	Cabin Class	First-Class
To	New York	Cabin Class	Economy
To	New York	Cabin Class	Premium Economy
To	New York	Cabin Class	Business
To	New York	Cabin Class	First-Class
To	London	Cabin Class	Economy
To	London	Cabin Class	Premium Economy
To	London	Cabin Class	Business
To	London	Cabin Class	First-Class

White box vs. Black box testing

Black-box testing

- Aka specification testing
- Test case generation are based solely on the knowledge of the system requirements.
- Ensures that specified features of the software are addressed by some aspect of the program

White-box testing

- Aka Code-based testing
- Test data selection is guided by information derived from the internal structure and internal functional properties of the program

```
if (testscore >= 90) {
    grade = 'A';
} else if (testscore >= 80) {
    grade = 'B';
} else if (testscore >= 70) {
    grade = 'C';
} else if (testscore >= 60) {
    grade = 'D';
} else {
    grade = 'F';
}
```

Statement coverage

- Test cases should cover all the executable statements
- Excluding comments, empty lines, etc.

```
examScore = int(input("Enter exam score: "))
isPass = False

if examScore >= 70:
    isPass = True
print(isPass)
```

Statement Coverage =

Test Case	examScore	attendance_rate	Expected Result
1	80	0.8	True

Decision (Branch) Coverage

- Derive test cases to cover all the decisions (branches) in the program.
- A decision/branch is covered if it evaluates to both true and false outcome by at least one test cases.

```
examScore = int(input("Enter exam score: "))
if examScore >= 70:
    isPass = True
else
    isPass = False
print(isPass)
```

Two decisions outcomes to be covered:
examScore: {T,F}

Decision Coverage =

Test Case	examScore	Expected Result
1		
2		

Multiple condition coverage

- The coverage domain consists of all the combinations of conditions in each decision.
- A decision with n conditions requires 2^n test cases!

```
examScore = int(input("Enter exam score: "))
attendance_rate = float(input("Enter attendance rate: "))

if  examScore >= 70 and attendance_rate>0.5:
    isPass = True
else
    isPass = False
print(isPass)
```

4 combinations of conditions
TT, TF, FT, FF

Multiple condition coverage =

Test Case	examScore	attendance_rate	Expected Result
1			
2			
3			
4			

Modified Condition/Decision (MC/DC) Coverage

Each simple condition within a compound condition C in P has been shown to independently effect the outcome of C.

```
examScore = int(input("Enter exam score: "))
attendance_rate = float(input("Enter attendance rate: "))

if  examScore >= 70 and attendance_rate>0.5:
    isPass = True
else
    isPass = False
print(isPass)
```

The decision: **`examScore >= 70`** and `attendance_rate>0.5`

To test the independent effect of the first condition, we set the second condition to be True.

Why? If we set the second condition to be false, the decision is false no matter what the first condition is True or false.

Test Case	examScore	attendance_rate	Expected Result
1	80	0.7	True
2	60	0.7	False

The decision: `examScore >= 70` and **`attendance_rate>0.5`**

Similarly, to test the independent effect of the second condition, we set the first condition to be True.

Why? If we set the first condition to be false, the decision is false no matter what the second condition is True or false.

Test Case	examScore	attendance_rate	Expected Result
~~3~~	~~80~~	~~0.7~~	~~True~~
4	80	0.4	False

Comparison of the coverage criteria based on control flow

- Condition, condition/decision, or decision coverage may not be able to reveal some common faults.
- Multiple condition coverage may reveal more faults, but it may generate too many test cases.
- MC/DC coverage is a weaker criterion than the multiple condition coverage criterion.
  - The number of test cases is much less than multiple condition coverage (in particular when there are many conditions in a decision), but it can detect most types of faults.

Loop Coverage

- Zero-pass : The loop is not executed at all.
- Single-pass : The loop is executed exactly once.
- Two-pass : The loop is executed exactly twice.
- Multi-pass : The loop is executed more than twice.

What are the test cases?

```
# input number of students
n = int(input("Enter number of students: "))

for i in range(n):
    # Input examScore
    examScore = int(input(f"Enter exam score for student {i+1}: "))
    attendance_rate = float(input(f"Enter attendance rate for student {i+1}: "))

    # Print whether the student pass/fail
    if examScore >= 70 and attendance_rate > 0.5:
        isPass = True
    else:
        isPass = False
    print(f"Student {i+1} passed? {isPass}")
```